SITE AUTOMATION

# Reports on a file share

Every report job's page written straight onto
a Windows file share over SMB - no mount

Site Automation - set up, test and use | site-automation 0.63.0 | October 2026

**What this is.** Every report job can write its report as a web page into a folder on a Windows file share, for example `\\fileserver\its\aap\reports`, straight over SMB: nothing mounted on the AAP execution nodes, no AAP setting, no web server that takes uploads. This guide sets it up from nothing, in order: the folder, the account, the execution environment, the network, the credential, the settings, a test - and your reports page.

**How to read it**
Steps 1 to 8 in order. The table at the end says what each message means. Rendered from `docs/REPORT_SHARE.md` in the site-automation repository (version 0.63.0); copy YAML from the repository, not from the PDF.

Host names and addresses in this guide are placeholders. Copy YAML from the repository files, not from the PDF.

# Contents

1. The reports on a file share (SMB) 3

What you get . . . 3
What you need . . . 3
Step 1. The folder and its permissions . . . 4
Step 2. The service account . . . 4
Step 3. The execution environment . . . 4
Step 4. The network path . . . 5
Step 5. The credential in AAP . . . 5
Step 6. The settings . . . 5
Step 7. Test it . . . 6
Step 8. The reports page (your portal) . . . 6
When something fails . . . 7
The other ways . . . 7

# 1. The reports on a file share (SMB)

Every report job can write its report as a web page into a folder on a Windows file share - for example `\\fileserver\its\aap\reports` - straight over SMB, the way Windows itself writes to a share. Nothing is mounted on the AAP execution nodes, no AAP setting changes, and no web server has to take uploads: the job logs in to the share with a service account and writes the files. A reports page (your site's portal, IIS, anything that reads that folder) then shows them.

This guide sets it up from nothing, in order. The report website itself - the names of the files, the index, the looks, the copy job - is in WEB_REPORTS.md.

## What you get

After each run of a report job, in the share's folder:

```
\\fileserver\its\aap\reports\
    index.html                          every report's last run, worst first (and index.json)
    linux-health-check\
        latest.html                     always this template's newest run (and latest.json)
        2026-10-26_0605_job-1234.html   each run's own page (and its .json)
    vm-health-report\
        latest.html
        ...
```

One folder per job template (its name in AAP, lower case with dashes). `latest.html` is the file to point a reports page's tile at: it is always the newest run. Each page stands alone - its logo is inside it, it needs no script and reaches nothing on the network.

The job prints where it wrote the page (`Published: \\fileserver\its\aap\reports\linux-health-check\...`), or its web address when you set one (`Report: https://...`). A run that cannot write stops with the reason in plain words.

## What you need

| Item | Example | Who gives it to you |
|---|---|---|
| A folder on the share | `\\fileserver\its\aap\reports` | you, or the storage team |
| A service account that may write there | `YOURDOMAIN\svc_aap_reports` | the Active Directory team |
| SMB (TCP 445) from the AAP execution nodes to the file server | | the network or firewall team |
| `smbprotocol` in the execution environment | in all three EEs of `ee/` since 0.32.0 | you (step 3) |
| The **Report website** credential type in AAP | `aap/credential_types/report_web.yml` | you (step 5) |

## Step 1. The folder and its permissions

1. Make the folder once: `\\fileserver\its\aap\reports`. The job never makes it - if it is not there (a typo, a share that moved), the job stops instead of writing somewhere else. It makes the folders inside it.
2. Give the service account (step 2) **Modify** on that folder (NTFS: *Properties* > *Security*) - Modify, not Write: the job also deletes older runs' pages (`report_web_history`, docs/WEB_REPORTS.md). On the share itself it needs at least **Change** (*Properties* > *Sharing* > *Advanced Sharing* > *Permissions*) - most shares give *Everyone: Change* or *Authenticated Users: Change* already and leave the rest to NTFS.
3. Give whoever reads the reports **Read** on the folder: the reports page's account (an IIS application pool identity, a service account), and the people who open the files.

**A mixed-mode (NFS and SMB) volume**, as on a NetApp: ask the storage team which security style the volume (or qtree) has. **NTFS** or **mixed** is what you want here: the files the job writes over SMB then take the folder's Windows permissions, and a Windows reader can always read them. With **UNIX** security style, the files get UNIX owners and modes from the SVM's name mapping - ask for NTFS, or test that the reports page can read a file the job wrote (step 7).

## Step 2. The service account

Ask the Active Directory team for an account only this uses:

```
Request: a service account for Ansible Automation Platform (AAP) to write report files to a share

- User name: svc_aap_reports (or your naming standard), in YOURDOMAIN
- Rights: Modify on \\fileserver\its\aap\reports (only there); no interactive logon (deny "Log on
  locally"
  and "Log on through Remote Desktop Services")
- Password: set, "User must change password at next logon" off. Tell me the expiry or rotation
  rule, so the
  AAP credential can be updated in time.
- Logon: SMB from the AAP execution nodes: [their names or addresses]
```

## Step 3. The execution environment

The job writes over SMB with **smbprotocol**, a pure-Python SMB 2/3 client (no compiler, no Samba). It is in all three execution environments of `ee/` (Windows, network, all-in-one) from release 0.32.0: build the ones your report templates use from those files or later, and update the image in AAP - EXECUTION_ENVIRONMENT.md, chapters 3 to 5 (one per EE) and "Put an EE into AAP". Check the new image:

```shell
podman run --rm site-ee-all:3 ee-check | grep -i smb
#   YES  smbprotocol
```

**Only one execution environment needs it** if you let the copy job write the pages (`report_web_copy: true`, WEB_REPORTS.md, "The copy job"): the report templates keep their own execution environments and credentials, each job keeps its report in AAP, and only the template **Report website - copy** - in the EE that has smbprotocol, with the credential of step 5 - writes them to the share.

A job in an execution environment without smbprotocol stops with: *this execution environment has no smbprotocol*.

## Step 4. The network path

TCP **445** from every AAP execution node that runs report jobs to the file server. Test it from a node:

```shell
timeout 3 bash -c '</dev/tcp/fileserver.yoursite.mil/445' && echo open || echo blocked
```

`blocked`: ask the network team for TCP 445 from those nodes to the file server.

## Step 5. The credential in AAP

1. **The credential type** (once): *Administration* > *Credential Types* > *Add*, name **Report website**, with the *Input configuration* and *Injector configuration* of `aap/credential_types/report_web.yml`. (It is the same type the WinRM and upload modes use: if you have it, skip this.)
2. **The credential**: *Automation Execution* > *Infrastructure* > *Credentials* > *Add*, type **Report website**, name e.g. `Report share`, **User name** `svc_aap_reports@YOURDOMAIN.MIL` - the account, `@`, and the domain's full name in capitals (its Kerberos realm) - and **Password**. Leave *Upload token* empty. `YOURDOMAIN\svc_aap_reports` works only with NTLM, never in a FIPS execution environment (step 6).
3. **Attach it** to each report template (*Templates* > *the template* > *Edit* > *Credentials*), next to its other credentials - or only to **Report website - copy**, with the copy job.

## Step 6. The settings

For every report job at once, in `playbooks/group_vars/all.yml` (the report jobs read them there, whatever their inventory):

single quotes: the backslashes stay; the server by its full name - Kerberos asks for a ticket to that name

```
report_web_dest: '\\fileserver.yourdomain.mil\its\aap\reports'
# negotiate (Kerberos, else NTLM) - leave it, FIPS too; ntlm; kerberos (only a ticket already in
  the job)
report_web_smb_auth: negotiate
# encrypt every byte (SMB 3); a share that requires it is encrypted anyway
# report_web_smb_encrypt: true
# only if the folder is also served at a web address: the emails then link it
# report_web_url: https://reports.yoursite.mil/aap
```

`smb://fileserver.yourdomain.mil/its/aap/reports` works too. Not in an inventory group's Variables: the page is written by the AAP side of the job, which does not get a group's variables. To try it on one template first, put the lines in that template's **Variables** box.

**FIPS.** Leave `report_web_smb_auth: negotiate`. It logs in with Kerberos first, with the credential's password - no ticket to set up - when the user name is `svc_aap_reports@YOURDOMAIN.MIL` (step 5) and `report_web_dest` names the server by its full name. NTLM, its fallback, needs the MD4 hash, which a FIPS-mode execution environment does not have: the message *NTLM needs MD4* means Kerberos did not happen - check those two. Not `kerberos`: it sends a bare Kerberos token, and a server that answers in SPNEGO - what `negotiate` speaks - stops it with *Invalid token was supplied*. The EEs of `ee/` have the Kerberos settings it needs; to try Kerberos from one by hand: EXECUTION_ENVIRONMENT.md, EE 1, "Step 4. Check Kerberos to Active Directory".

## Step 7. Test it

1. Launch one report template - **VM - health report**, say - with **Job type: Check**. Check writes nothing, but it logs in and looks at the folder. **You should see:** `Dry run (Check): logged in to fileserver, the folder \\fileserver\its\aap\reports is there; the report would be published at ...`
2. Launch it for real. **You should see:** `Published: \\fileserver\its\aap\reports\vm-health-report\2026-..._job-N.html`.
3. Open the folder in Windows Explorer: the template's folder with `latest.html`, and `index.html`. Open them in a browser.
4. As the reports page's account (or a colleague), open `latest.html` too: proof that the readers can read what the job wrote (step 1, mixed-mode volumes).

## Step 8. The reports page (your portal)

1. Its report folder: `\\fileserver\its\aap\reports` (for a portal with a `reportFolders` setting in its configuration, that path).
2. One tile per report, pointing at that template's `latest.html`, e.g. `\\fileserver\its\aap\reports\vm-health-report\latest.html` - always the newest run. `index.html` makes a good "every report" tile: every template's last run, worst first, each linked.
3. A live miniature of the page is fine (the pages are plain HTML). The pages need **no scripts**: leave "run the report's own scripts" off.
4. A tile appears once the template has run once with the settings of step 6.

## When something fails

| The job says | Why | Do this |
|---|---|---|
| `this execution environment has no smbprotocol` | the template's EE is older than 0.33.0's all-in-one | build and use the new EE (step 3), or the copy job |
| `no login for the share ...: attach the "Report website" credential` | the template has no Report website credential | step 5.3 |
| `the user name or password is wrong ... or the account is locked` | the login was refused | the credential's user name (`svc_aap_reports@YOURDOMAIN.MIL`) and password; is the account locked or expired? |
| `Configuration file does not specify default realm` | the user name has no realm (`YOURDOMAIN\user`, or no domain) | `svc_aap_reports@YOURDOMAIN.MIL`, the realm in capitals (step 5.2) |
| `Server not found in Kerberos database` | the server's short name in `report_web_dest` | its full name: `\\fileserver.yourdomain.mil\its\...` (step 6) |
| `Invalid token was supplied` | `report_web_smb_auth: kerberos` | `negotiate` (step 6, FIPS) |
| `the account may log in but not use this folder` | access denied | step 1: Modify on the folder, Change on the share |
| `no such share on that server` | the part after the server name is wrong | the share's name: `\\fileserver\its`, as Explorer shows it |
| `the folder ... is not there - make it once by hand` | the folder under the share is missing | make it (step 1), or correct `report_web_dest` |
| `cannot be reached on port 445` | firewall, routing or the name | step 4 |
| `NTLM needs MD4 ... Kerberos did not happen` | a FIPS execution environment, and Kerberos did not log in | step 6, FIPS: `negotiate`, `user@YOURDOMAIN.MIL`, the server's full name |
| The page is there, but the reports page cannot open it | the readers lack Read, or a UNIX-style volume | step 1 |

## The other ways

The same pages, written another way - pick one (WEB_REPORTS.md, "Where the files go"):

- **A share mounted on the execution nodes** (`report_web_dest: /mnt/reports/aap`): an NFS or SMB mount on every node, exposed to the jobs (*Settings* > *Job settings* > *Paths to expose to isolated jobs*). No credential in the job, no smbprotocol; root on the nodes and the same mount on each.
- **A Windows server over WinRM** (`report_web_host`): the pages written to a folder **on** that server (`D:\reports`) - not a share, which would be a second hop.
- **A web server that takes uploads** (HTTP PUT, WebDAV).